ABDULLAH HASEEB ABBASI

SP25-BCS-060

DATA STRUCTURE (LAB ASSIGHMENT-01)

SUBMIT TO :Mr Bilal Bukhari

Problem Statement

This program implements an ArrayList using a struct with a fixed capacity and a logical size. A traversal pointer (ptr) is used to walk through the logical elements, while minPtr, maxPtr, medianPtr and closestPtr are made to point to the corresponding elements inside the original ArrayList, as required by the assignment.

Source Code (C++)

```
#include <iostream>
#include <cmath>
using namespace std;

const int CAPACITY = 20;

struct ArrayList {
    int data[CAPACITY];
    int size = 0;
};

bool insertEnd(ArrayList &l, int v) {
    if (l.size == CAPACITY) return false;
    l.data[l.size++] = v;
    return true;
}

bool insertAtBeginning(ArrayList &l, int v) {
    if (l.size == CAPACITY) return false;
    for (int i = l.size; i > 0; i--)
        l.data[i] = l.data[i - 1];
    l.data[0] = v;
    l.size++;
    return true;
}

bool deleteAtPosition(ArrayList &l, int p) {
    if (p < 0 || p >= l.size) return false;
    for (int i = p; i < l.size - 1; i++)
        l.data[i] = l.data[i + 1];
```

```
    l.size--;
    return true;
}

void displayList(const ArrayList &l) {
    for (int i = 0; i < l.size; i++)
        cout << l.data[i] << " ";
    cout << endl;
}

int main() {
    ArrayList l;
    int *ptr, *minPtr, *maxPtr, *medianPtr, *closestPtr;
    int sum = 0, closestPosition;

    insertEnd(l, 18);
    insertEnd(l, 7);
    insertEnd(l, 45);
    insertEnd(l, 11);
    insertEnd(l, 36);
    insertEnd(l, 60);
    insertEnd(l, 21);
    insertEnd(l, 13);
    insertEnd(l, 29);

    cout << "Initial ArrayList: ";
    displayList(l);

    ptr = minPtr = maxPtr = l.data;

    for (int i = 0; i < l.size; i++, ptr++) {
        sum += *ptr;
        if (*ptr < *minPtr) minPtr = ptr;
        if (*ptr > *maxPtr) maxPtr = ptr;
    }

    int temp[CAPACITY];

    for (int i = 0; i < l.size; i++)
        temp[i] = l.data[i];

    for (int i = 0; i < l.size - 1; i++)
        for (int j = 0; j < l.size - i - 1; j++)
            if (temp[j] > temp[j + 1])
                swap(temp[j], temp[j + 1]);
```

```
int median = temp[l.size / 2];
medianPtr = l.data;

for (int i = 0; i < l.size; i++, medianPtr++)
   if (*medianPtr == median) break;

double generalAverage = (double)sum / l.size;
double specialAverage = (*minPtr + *medianPtr + *maxPtr) / 3.0;

closestPtr = l.data;
double d = fabs(*closestPtr - specialAverage);

for (ptr = l.data; ptr < l.data + l.size; ptr++) {
   if (fabs(*ptr - specialAverage) < d) {
      d = fabs(*ptr - specialAverage);
      closestPtr = ptr;
   }
}

closestPosition = closestPtr - l.data;

double difference = fabs(generalAverage - specialAverage);
double finalScore = fabs(*closestPtr - generalAverage)
               + fabs(*closestPtr - specialAverage)
               + difference;

cout << "Minimum Value: " << *minPtr << endl;
cout << "Maximum Value: " << *maxPtr << endl;
cout << "Sum: " << sum << endl;
cout << "Median Value: " << *medianPtr << endl;
cout << "General Average: " << generalAverage << endl;
cout << "Special Average: " << specialAverage << endl;
cout << "Closest Value: " << *closestPtr << endl;
cout << "Position of Closest Value: " << closestPosition << endl;
cout << "Difference Between Averages: " << difference << endl;
cout << "Final Score: " << finalScore << endl;

deleteAtPosition(l, closestPosition);

cout << "ArrayList After Deletion: ";
displayList(l);

insertAtBeginning(l, round(specialAverage));

cout << "Final ArrayList After Insertion: ";
displayList(l);
```

```
    return 0;

}}
```

Program Output

```
Initial ArrayList: 18 7 45 11 36 60 21 13 29
Minimum Value: 7
Maximum Value: 60
Sum: 240
Median Value: 21
General Average: 26.6667
Special Average: 29.3333
Closest Value: 29
Position of Closest Value: 8
Difference Between Averages: 2.66667
Final Score: 5.33333
ArrayList After Deletion: 18 7 45 11 36 60 21 13
Final ArrayList After Insertion: 29 18 7 45 11 36 60 21 13
```